

SmartShelf

Smart Retail Inventory & First-Expiry-First-Out (FEFO) Management System

TEAM REPORT - PART 1 of 4 | Team Member 1: [Your Name]

Your role on the project: Project Lead & opening presenter

What YOU present and defend: The 'what and why' of SmartShelf, the problem it solves, the product idea, the architecture, the tech stack and how to run the demo.

How to use this part

Read your sections below until you can walk someone through them without notes. Then practise the Q&A cues at the end - they mirror what a panel typically asks. If you do not know a detail outside your part, say so and hand it to the right teammate.

You must be able to explain

- The one-sentence description of the project.
- The retail-shrink problem and why FEFO is the answer.
- The layered architecture and each layer's job.
- The technology stack and the databases supported.
- How to run the app and the demo logins.

1. The project in one sentence

SmartShelf is a web-based inventory management system for retail stores selling perishable, dated products. Every unit of stock belongs to a dated batch, and the system sells, discounts and alerts based on expiry dates compared against how fast items sell.

2. The problem it solves

- Roughly 20-30% of perishable stock is lost to expiry before it can be sold.
- Without FEFO, staff sell the newest stock they grab and the oldest stock rots.
- Discounts were gut-feel; nobody could prove which batch was sold or how much waste cost.

The business outcome SmartShelf targets: **sell the oldest stock first automatically, quantify spoilage risk, and approve discounts deliberately** - so waste falls and margin holds.

3. Solution overview and capabilities

- Real-time batch-level stock with cost, value, expiry and location.
- Automatic FEFO picking at every sale (never a manual first-out decision).
- Risk-scored batches and recommended discount percent with a clear reason.
- 30-day expiry alerts, low-stock and high-risk notifications.
- Role-based access for Admin, Manager, Biller, Cashier, Staff and User.
- Full audit trail: every sale, promotion, adjustment and waste event is logged.

4. The product lifecycle

5 stages: **Receive** (purchase receipts create dated batches), **Store** (locations, value, alerts), **Monitor** (expiry + risk scoring), **Sell** (FEFO-first checkout), **Analyze** (velocity, waste, margin) - then the cycle repeats with better data.

5. Reference architecture

SmartShelf - Smart Retail Inventory & FEFO Management System

Reference Architecture & End-to-End Working Flow

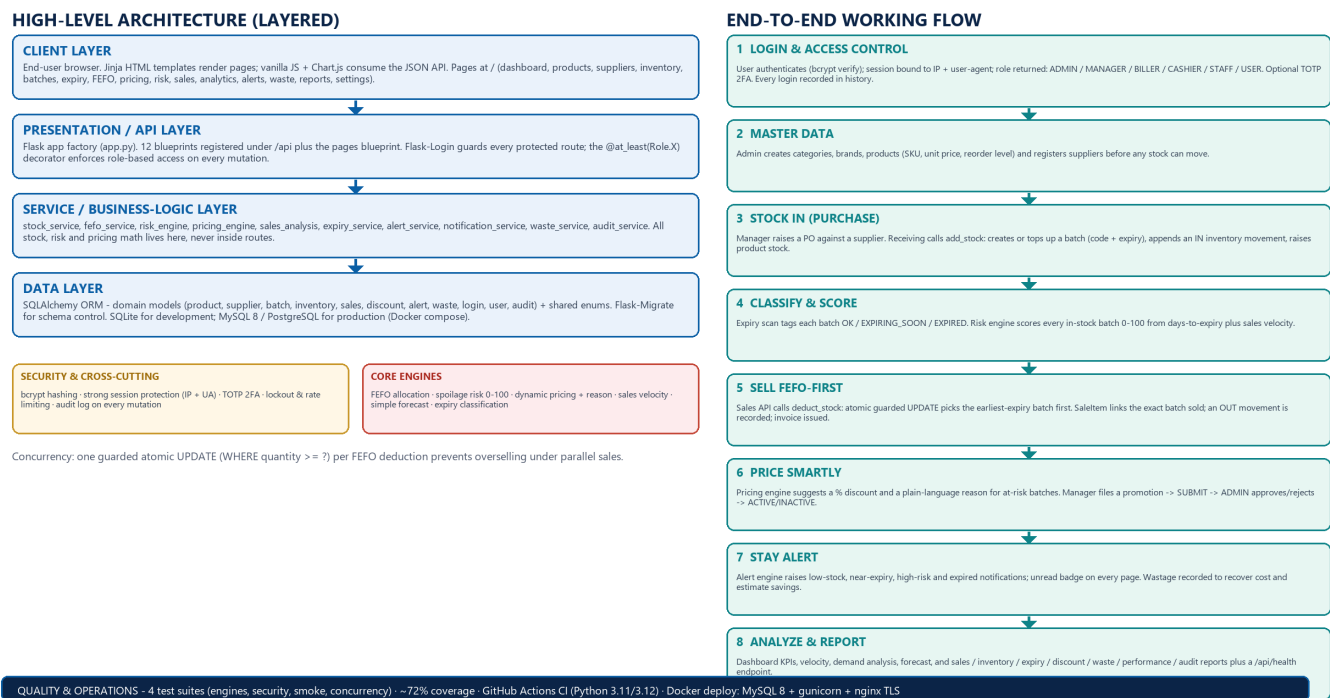


Figure: layered architecture (client -> API -> services/engines -> data) plus the 8-step end-to-end working flow.

6. Technology stack

Layer	Technology	Why
Backend	Python 3 + Flask + SQLAlchemy	Lightweight blueprints, clean REST JSON API
Frontend	HTML/CSS/JS + Chart.js	Server-rendered pages, live dashboard charts
Database	SQLite (dev) / MySQL / PostgreSQL (prod)	DATABASE_URL switch, ORM everywhere
Security	Flask-Login, bcrypt, itsdangerous	Sessions, TOTP 2FA, signed reset tokens
Deploy	Docker + gunicorn + nginx TLS	One-command compose, healthchecks

7. How to run the demo

- .venv\Scripts\python.exe -m flask --app app init-db (create schema)
- .venv\Scripts\python.exe -m flask --app app seed (demo data)
- .venv\Scripts\python.exe -m flask --app app run (http://localhost:5000)
- Demo logins: admin/admin123 manager/manager123 cashier/cashier123

8. Likely questions and answer cues

Q. Describe the project in one sentence.

A. A retail inventory system that dates every batch and uses FEFO so the store sells the oldest stock first, scores spoilage risk and approves discounts before loss.

Q. Who is the user, and what is the main pain point?

A. A store owner/manager losing money to expired stock. They need to know what is about to expire, sell it first and discount it deliberately.

Q. Walk me through the architecture.

A. Browser pages call a Flask REST API. Routes delegate to service/engine modules that do FEFO, risk and pricing math, then ORM writes to SQLite/MySQL/PostgreSQL. Sessions use Flask-Login, mutations are audited.

Q. Why is FEFO better than FIFO for perishables?

A. FIFO orders by arrival; FEFO orders by expiry date. A newer batch can expire sooner, so expiry priority is the only safe rule for food.

Q. Which databases do you support and how do you switch?

A. SQLite for development, MySQL and PostgreSQL for production, selected through the DATABASE_URL environment variable - the ORM hides the difference.

Q. How do I verify the app works?

A. Log in with admin/admin123, receive stock into a batch, sell and watch the FEFO pick, open Pricing and Analytics; the whole journey is covered by automated tests too.